

MI IPU API

Version 3.0

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	• Initial release	07/23/2021
	• Added PROCFS introduction	08/25/2021
	• Modify default ipu firmware file path	11/30/2021
	• Modify parameter(pReadCtx) description of MI_IPU_CreateCHN	
	• Modify maximum channel number	
	• Modify the usage of tensor shape	
	• Modify maximum tensor dimension	
	• Modify error codes	
	• Modify ipu frequency path on /sys	
	• Add swdisp related error codes	12/21/2021
	• Add u64BandWidthRead/u64BandWidthWrite to MI_IPU_RuntimeInfo_t structure	
	• Add ipu buffer alignment error code	
	• Modify IPU buffer alignment number	01/20/2022
	• Add MI_IPU_Invoke2Custom function	02/25/2022
	• Modify error codes	
	• Add MI_IPU_FORMAT_GRAY tensor format	
	• Add member 'u32BufSize', 'u32InputWidthAlignment', 'u32InputHeightAlignment' and 'bOutputNCHW' in MI_IPU_TensorDesc_t	

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. API reference	1
1.1. Summary	1
1.2. Keyword Description	1
1.3. MI IPU API	2
1.3.1 MI_IPU_CreateDevice	3
1.3.2 MI_IPU_DestroyDevice	4
1.3.3 MI_IPU_CreateCHN	4
1.3.4 MI_IPU_DestroyCHN	6
1.3.5 MI_IPU_GetInOutTensorDesc	6
1.3.6 MI_IPU_GetInputTensors	8
1.3.7 MI_IPU_PutInputTensors	9
1.3.8 MI_IPU_GetOutputTensors	10
1.3.9 MI_IPU_PutOutputTensors	11
1.3.10 MI_IPU_Invoke	12
1.3.11 MI_IPU_GetInputTensors2	13
1.3.12 MI_IPU_PutInputTensors2	13
1.3.13 MI_IPU_GetOutputTensors2	14
1.3.14 MI_IPU_PutOutputTensors2	15
1.3.15 MI_IPU_Invoke2	15
1.3.16 MI_IPU_Invoke2Custom	17
1.3.17 MI_IPU_GetOfflineModeStaticInfo	20
2. Data types	22
2.1. SerializedReadFunc	23
2.2. MI_IPU_ELEMENT_FORMAT	23
2.3. MI_IPU_TensorDesc_t	24
2.4. MI_IPU_SubNet_InputOutputDesc_t	26
2.5. MI_IPU_Tensor_t	26
2.6. MI_IPU_TensorVector_t	27
2.7. MI_IPU_DevAttr_t	28
2.8. MI_IPU_ChnAttr_t	29
2.9. MI_IPU_BatchInvoke_param_t	29
2.10. MI_IPU_RuntimeInfo_t	30
2.11. MI_IPU_OfflineModelStaticInfo_t	31
3. Error codes	32
4. PROCFS INTRODUCTION	34
4.1. Summary	34
4.2. How to cat MMA info	34
4.3. Cat IPU version info	35
4.4. Cat IPU clock info	35
4.5. Adjust IPU clock	36
4.6. Switch auto reset function	36

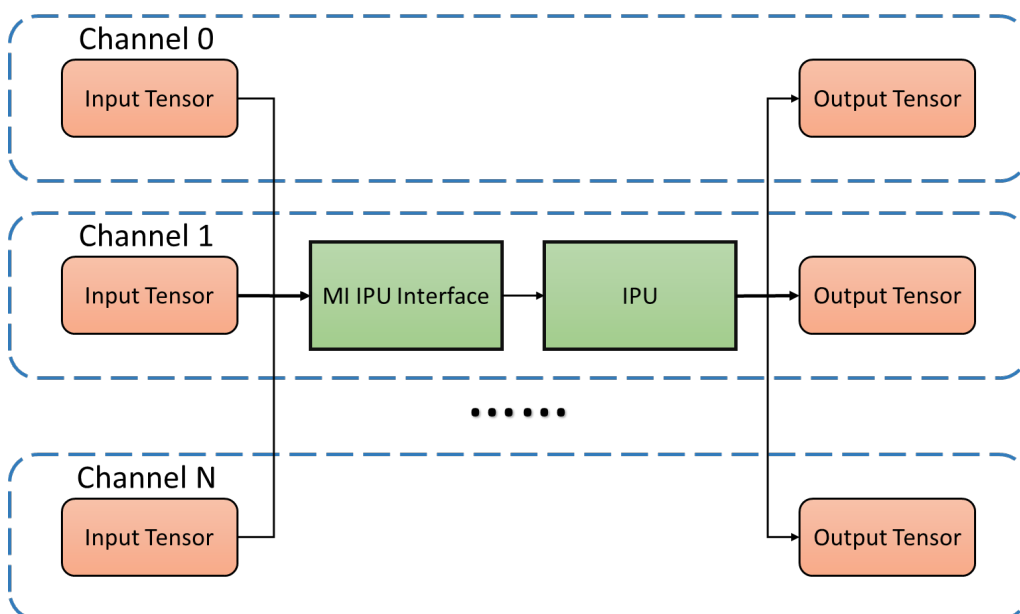
4.7. Get IPU log.....	37
-----------------------	----

1. API REFERENCE

1.1. Summary

MI IPU module accelerates the inferences of AI models, and supports multi AI models by channels. It Internally manages the acquisition and release of input and output Tensor while supporting external buffer as Input Tensor.

Putting flow chart



1.2. Keyword Description

- IPU
Intelligent process unit
- Firmware
The program which drives the IPU HW
- Tensor
Multi dimension data in the AI model
- Input Tensor
Input Tensor of the AI model
- Output Tensor
Output Tensor of the AI model

1.3. MI IPU API

API	Function
MI_IPU_CreateDevice	Create an IPU device
MI_IPU_DestroyDevice	Destroy an IPU device
MI_IPU_CreateCHN	Create an IPU channel
MI_IPU_DestroyCHN	Destroy an IPU channel
MI_IPU_GetInOutTensorDesc	Get the input and output information of the network by specified channel
MI_IPU_GetInputTensors	Allocate specified channel for input Tensor Buffer
MI_IPU_PutInputTensors	Release specified channel for input Tensor Buffer
MI_IPU_GetOutputTensors	Allocate specified channel for output Tensor Buffer
MI_IPU_PutOutputTensors	Release specified channel for output Tensor Buffer
MI_IPU_Invoke	Execute AI network inference
MI_IPU_GetInputTensors2	Get batch input Tensor Buffer from the specified channel
MI_IPU_PutInputTensors2	Release the batch input Tensor Buffer of the specified channel
MI_IPU_GetOutputTensors2	Get batch output Tensor Buffer from the specified channel
MI_IPU_PutOutputTensors2	Release the batch output Tensor Buffer of the specified channel
MI_IPU_Invoke2	Execute n_buf batch AI network inference
MI_IPU_Invoke2Custom	Execute one_buf batch AI network inference
MI_IPU_GetOfflineModeStaticInfo	Get variable buffer size and offline model file size of an offline model

1.3.1 MI_IPU_CreateDevice

➤ Function

Create an IPU device.

➤ Definition

```
MI_S32 MI_IPU_CreateDevice(MI_IPU_DevAttr_t *pstIPUDevAttr, SerializedReadFunc
pReadFunc, char *pReadCtx, MI_U32 FWSize);
```

➤ Parameter

Parameters	Description	Input/Output
pstIPUDevAttr	IPU device attributes structure pointer	Input
pReadFunc	User's custom function pointer to read file (using default file reading function by MI IPU if set NULL)	Input
pReadCtx	IPU firmware path (default is "/config/dla/ipu_firmware.bin" by MI IPU if set NULL)	Input
FWSize	IPU firmware file size (default is auto get file size if set 0)	Input

➤ Return value

- MI_SUCCESS: Successful
- Others: Failed, see [error code](#) for details

➤ Dependence

- Head file: mi_ipu.h
- Library: libmi_ipu.so

➤ Note

- None

➤ Example

```
MI_S32 s32Ret;

stDevAttr.u32MaxVariableBufSize = BufSize; /* The maximum size of the memory used by
Tensor in the model */

s32Ret = MI_IPU_CreateDevice(&stDevAttr, NULL, NULL, 0);
if (s32Ret != MI_SUCCESS) {
    printf("fail to create ipu device\n");
    return s32Ret;
}
```

➤ Related subject

None.

1.3.2 MI_IPU_DestroyDevice

- **Function**
Destroy an IPU device.
- **Definition**
`MI_S32 MI_IPU_DestroyDevice(void);`
- **Parameter**
None.
- **Return value**
 - MI_SUCCESS: Successful
 - Others: Failed, see [error code for](#) details
- **Dependence**
 - Head file: `mi_ipu.h`
 - Library: `libmi_ipu.so`
- **Note**
 - None
- **Example**
`MI_IPU_DestroyDevice();`
- **Related subject**
None.

1.3.3 MI_IPU_CreateCHN

- **Function**
Create an IPU channel.
- **Definition**
`MI_S32 MI_IPU_CreateCHN(MI_IPU_CHN *ptChnId, MI\_IPUChnAttr\_t *pstIPUChnAttr, SerializedReadFunc pReadFunc, char *pReadCtx);`

- **Parameter**

Parameters	Description	Input/Ouput
ptChnId	The point of created IPU channel ID	Input
pstIPUChnAttr	IPU channel attributes structure pointer	Input
pReadFunc	User's custom function pointer to read file (using default	Input

Parameters	Description	Input/Ouput
	file reading function by MI IPU if set NULL)	
pReadCtx	AI Network file path or file memory address of OS	Input

➤ Return value

- MI_SUCCESS: Successful
- Others: Failed, see [error code for](#) details

➤ Dependence

- Head file: mi_ipu.h
- Library: libmi_ipu.so

➤ Note

- MI_IPUChnAttr_t.u32BatchMax is the maximum value of batch processing. If not needed, set this value to 1 or 0.
- MI_IPUChnAttr.u32InputBufDepth is the number of pre allocated private input tensor buffer* u32BatchMax, such as 0 or 1 or 2 or 3. When set to 0, it represents the buffer allocated from external module. If the output buffers of previous MI modules can be directly used, it is better to set input_depth to zero for memory saving.
- MI_IPUChnAttr_t. u32OutputBufDepth is the number of pre allocated private output tensor buffer* u32BatchMax, such as 0 or 1 or 2 or 3. When set to 0, it represents the buffer allocated from external module, such as the MI_RGN module.
- If user creates channel with one_buf batch model, MI IPU won't pre allocate input/output tensor buffers.
- The maximum number of IPU channels is 48.

➤ Example

```
MI_S32 s32Ret, buf_depth = 3, batch_max = 1;
MI_IPU_CHN u32ChnId = 0;
chnAttr.u32InputBufDepth = buf_depth;
chnAttr.u32OutputBufDepth = buf_depth;
chnAttr. u32BatchMax = batch_max;
char pReadCtx[] = "caffe_mobilenet_v2.tflite_sgsmg.img";
s32Ret = MI\_IPU\_CreateCHN(&u32ChnId, &chnAttr, NULL, pReadCtx);
if (s32Ret != MI_SUCCESS) {
    printf("fail to create ipu channel\n");
    MI_IPU_DestroyDevice();
    return s32Ret;
}
```

- Related subject
None.

1.3.4 MI_IPU_DestroyCHN

- Function
Destroy an IPU channel.
- Definition
MI_S32 MI_IPU_DestroyCHN(MI_IPU_CHN u32ChnId);

- Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input

- Return value
 - MI_SUCCESS: Successful
 - Others: Failed, see [error code for](#) details
- Dependence
 - Head file: mi_ipu.h
 - Library: libmi_ipu.so
- Note
 - The maximum number of IPU channels is 48.
- Example
MI_IPU_DestroyCHN(u32ChnId);
- Related subject
None.

1.3.5 MI_IPU_GetInOutTensorDesc

- Function
Get the input and output information of the network by specified channel.
- Definition
MI_S32 MI_IPU_GetInOutTensorDesc(MI_IPU_CHN u32ChnId,
[MI_IPU_SubNet_InputOutputDesc_t](#) *pstDesc);
- Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstDesc	Network output description structure pointer	Output

- Return value
 - MI_SUCCESS: Successful
 - Others: Failed, see [error code for](#) details
- Dependence
 - Head file: mi_ipu.h
 - Library: libmi_ipu.so
- Note
 - The maximum number of IPU channels is 48.

➤ Example

```
MI_IPU_SubNet_InputOutputDesc_t desc;
s32Ret = MI\_IPU\_GetInOutTensorDesc(u32ChnId, &desc);
if (s32Ret) {
    printf("fail to get network description\n");
    MI\_IPU\_DestroyCHN(u32ChnId);
    MI\_IPU\_DestroyDevice();
    return s32Ret;
}
else {
    int iNum = desc.stMI_InputTensorShapes[0].u32TensorShape[0];
    int iResizeH = desc.stMI_InputTensorShapes[0].u32TensorShape[1];
    int iResizeW = desc.stMI_InputTensorShapes[0].u32TensorShape[2];
    int iResizeC = desc.stMI_InputTensorShapes[0].u32TensorShape[3];
    unsigned char *pu8ImageData = new unsigned char[iNum*iResizeH*iResizeW*iResizeC];
    ...
}
```

- Related subject

None.

1.3.6 MI_IPU_GetInputTensors

➤ Function

Allocate specified channel for input Tensor Buffer.

➤ Definition

```
MI_S32 MI_IPU_GetInputTensors(MI_IPU_CHN u32ChnId, MI_IPU_TensorVector_t
*pstInputTensorVector);
```

➤ Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstInputTensorVector	Input IPU Tensor array structure pointer	Input

➤ Return value

- MI_SUCCESS: Successful
- Others: Failed, see [error code for](#) details

➤ Dependence

- Head file: mi_ipu.h
- Library: libmi_ipu.so

➤ Note

- There are two ways to input the buffer source of tensor:
 1. Allocate buffer through MI_IPU_GetInputTensors;
 2. Allocated buffer by other modules of MI.

➤ Example

allocate input tensor by [MI_IPU_GetInputTensors](#)

```
MI_IPU_TensorVector_t inputV;
MI_IPU_TensorVector_t OutputV;
MI_U8 au8MaggiePic[3*224*224] ;
cv::Mat img = cv::imread(filename, -1);
cv::Size inputSize = cv::Size(224,224,3)
cv ::Mat imgResize ;
cv::resize(img, imgResize, inputSize);
memcpy(au8MaggiePic, imgResize.data, sizeof(au8MaggiePic));
s32Ret = MI_IPU_GetInputTensors(u32ChnId, &inputV);
if (s32Ret == MI_SUCCESS) {
    memcpy(inputV.stArrayTensors[0].pstTensorData[0], au8MaggiePic, sizeof(au8MaggiePic));
    MI_SYS_FlushInvCache(inputV.stArrayTensors[0].pstTensorData[0], sizeof(au8MaggiePic));
}
else {
```

```

    printf("fail to get buffer, please try again");
}
MI_IPU_GetOutputTensors( u32ChannelID, &OutputV);
if(MI_SUCCESS!=MI_IPU_Invoke(u32ChannelID, & inputV, &OutputV))
{
    cout<<"IPU invoke failed!!"<<endl;

    MI_IPU_DestroyCHN(u32ChannelID);
    MI_IPU_DestroyDevice();
    return -1;
}

```

use external buffer as Input tensor

```

MI_IPU_TensorVector_t inputVector, outputVector;
inputVector.u32TensorCount = 1;
if (stBufInfo.eBufType == E_MI_SYS_BUFDATA_RAW) {
    inputVector.astArrayTensors[0].phyTensorAddr[0] = stBufInfo.stRawData.phyAddr;
    inputVector.astArrayTensors[0].ptTensorData[0] = stBufInfo.stRawData.pVirAddr;
} else if (stBufInfo.eBufType == E_MI_SYS_BUFDATA_FRAME) {

    inputVector.astArrayTensors[0].phyTensorAddr[0] = stBufInfo.stFrameData.phyAddr[0];
    inputVector.astArrayTensors[0].ptTensorData[0] = stBufInfo.stFrameData.pVirAddr[0];

    inputVector.astArrayTensors[0].phyTensorAddr[1] = stBufInfo.stFrameData.phyAddr[1];
    inputVector.astArrayTensors[0].ptTensorData[1] = stBufInfo.stFrameData.pVirAddr[1];
}

//prepare output vector
s32Ret = MI_IPU_GetOutputTensors(FdaChn, &outputVector);
s32Ret = MI_IPU_Invoke(FdaChn, &inputVector, &outputVector);

```

- Related subject
None.

1.3.7 MI_IPU_PutInputTensors

- Function
Release specified channel for input Tensor Buffer.
- Definition
MI_S32 MI_IPU_PutInputTensors(MI_IPU_CHN u32ChnId,
MI_IPU_TensorVector_t *pstInputTensorVector);

➤ Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstInputTensorVector	Input IPU Tensor array structure pointer	Output

➤ Return value

- MI_SUCCESS: Successful
- Others: Failed, see [error code for](#) details

➤ Dependence

- Head file: mi_ipu.h
- Library: libmi_ipu.so

➤ Note

- None

➤ Example

None.

➤ Related subject

None.

1.3.8 MI_IPU_GetOutputTensors

➤ Function

Allocate specified channel for output Tensor Buffer.

➤ Definition

MI_S32 MI_IPU_GetOutputTensors(MI_IPU_CHN u32ChnId, [MI_IPU_TensorVector_t](#) *pstInputTensorVector);

➤ Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstInputTensorVector	Output IPU Tensor array structure pointer	Output

➤ Return value

- MI_SUCCESS: Successful
- Others: Failed, see [error code for](#) details

➤ Dependence

- Head file: mi_ipu.h
- Library: libmi_ipu.so

➤ Note

•

➤ Example

```
MI_IPU_TensorVector_t outputV;
s32Ret = MI_IPU_GetOutputTensors(u32ChnId, &outputV);
if (s32Ret != MI_SUCCESS) {
    printf("fail to get buffer, please try again");
}
```

➤ Related subject

None.

1.3.9 MI_IPU_PutOutputTensors

➤ Function

Release specified channel for output Tensor Buffer.

➤ Definition

```
MI_S32 MI_IPU_PutOutputTensors(MI_IPU_CHN u32ChnId, MI_IPU_TensorVector_t
*pstOutputTensorVector);
```

➤ Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstOutputTensorVector	Output IPU Tensor array structure pointer	Input

➤ Return value

- MI_SUCCESS: Successful
- Others: Failed, see [error code for](#) details

➤ Dependence

- Head file: mi_ipu.h
- Library: libmi_ipu.so

➤ Note

•

➤ Example

None.

- Related subject
None.

1.3.10 MI_IPU_Invoke

- Function
Execute AI network inference.

- Definition
MI_S32 MI_IPU_Invoke(MI_IPU_CHN u32ChnId, [MI_IPU_TensorVector_t](#) *pstInputTensorVector, [MI_IPU_TensorVector_t](#) *pstOutputTensorVector);

- Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstInputTensorVector	Input IPU Tensor array structure pointer	Input
pstOutputTensorVector	Output IPU Tensor array structure pointer	Input

- Return value
 - MI_SUCCESS: Successful
 - Others: Failed, see [error code for](#) details
- Dependence
 - Head file: mi_ipu.h
 - Library: libmi_ipu.so
- Note
 - MI_IPU_Invoke is synchronous api.
 - The start physical address of each single input/output tensor must be 64 bytes aligned.
- Example


```
s32Ret = MI\_IPU\_Invoke(u32ChnId, &inputV, &outputV);
if (s32Ret == MI_SUCCESS) {
    // process output buffer data
    // ...
}
```
- Related subject
None.

1.3.11 MI_IPU_GetInputTensors2

➤ Function

Get batch input Tensor Buffer from the specified channel.

➤ Definition

MI_S32 MI_IPU_GetInputTensors2(MI_IPU_CHN u32ChnId, [MI_IPU_BatchInvoke_param_t](#) *pstInvokeParam);

➤ Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstInvokeParam	Batch parameter structure pointer	Input

➤ Return value

- MI_SUCCESS: Successful
- Others: Failed, see [error code for](#) details

➤ Dependence

- Head file: mi_ipu.h
- Library: libmi_ipu.so

➤ Note

The source of Tensor's Buffer:

1. Allocate Buffer through MI_IPU_GetInputTensors2;
2. Use the buffer allocated by other modules of MI.

➤ Example

None.

➤ Related subject

None.

1.3.12 MI_IPU_PutInputTensors2

➤ Function

Release the batch input Tensor Buffer of the specified channel.

➤ Definition

MI_S32 MI_IPU_PutInputTensors2(MI_IPU_CHN u32ChnId, [MI_IPU_BatchInvoke_param_t](#) *pstInvokeParam);

➤ Parameter

Parameters	Description	Input/Output
------------	-------------	--------------

u32ChnId	IPU channel ID	Input
pstInvokeParam	Batch parameter structure pointer	Input

- Return value
 - MI_SUCCESS: Successful
 - Others: Failed, see [error code for](#) details
- Dependence
 - Head file: mi_ipu.h
 - Library: libmi_ipu.so
- Note

None.
- Example

None.
- Related subject

None.

1.3.13 MI_IPU_GetOutputTensors2

- Function

Get batch output Tensor Buffer from the specified channel.
- Definition

MI_S32 MI_IPU_PutInputTensors2(MI_IPU_CHN u32ChnId, [MI_IPU_BatchInvoke_param_t](#) *pstInvokeParam);

- Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstInvokeParam	Batch parameter structure pointer	Input

- Return value
 - MI_SUCCESS: Successful
 - Others: Failed, see [error code for](#) details
- Dependence
 - Head file: mi_ipu.h
 - Library: libmi_ipu.so
- Note

None.

➤ Example

None.

➤ Related subject

None.

1.3.14 MI_IPU_PutOutputTensors2

➤ Function

Release the batch output Tensor Buffer of the specified channel.

➤ Definition

MI_S32 MI_IPU_PutOutputTensors2(MI_IPU_CHN u32ChnId, [MI_IPU_BatchInvoke_param_t](#) *pstInvokeParam);

➤ Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstInvokeParam	Batch parameter structure pointer	Input

➤ Return value

- MI_SUCCESS: Successful
- Others: Failed, see [error code for](#) details

➤ Dependence

- Head file: mi_ipu.h
- Library: libmi_ipu.so

➤ Note

None.

➤ Example

None.

➤ Related subject

None.

1.3.15 MI_IPU_Invoke2

➤ Function

Execute n_buf batch AI network inference.

➤ Definition

```
MI_S32 MI_IPU_Invoke2(MI_IPU_CHN u32ChnId, MI_IPU_BatchInvoke_param_t
*pstInvokeParam, MI_IPU_RuntimeInfo_t *pstRuntimeInfo);
```

➤ Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstInvokeParam	Batch parameter structure pointer	Input
pstRuntimeInfo	IPU operation information structure pointer	Input

➤ Return value

- MI_SUCCESS: Successful
- Others: Failed, see [error code for](#) details

➤ Dependence

- Head file: mi_ipu.h
- Library: libmi_ipu.so

➤ Note

- MI_IPU_Invoke2 is synchronous api.
- MI_IPU_Invoke2 can only be used with n_buf mode model.
- The start physical address of each single input/output tensor must be 64 bytes aligned.
- The start physical address of variable tensor which is allocated by user must be 64 bytes aligned.

➤ Example

```
MI_IPU_BatchInvoke_param_t stInvokeParam;
MI_IPU_RuntimeInfo_t stRuntimeInfo;
stInvokeParam.u32BatchN = 16;
stInvokeParam.s32TaskPrio = 20;
stInvokeParam.u32IpuAffinity = 0; // Called by ipu
s32Ret = MI_IPU_GetInputTensors2(u32ChnId, &stInvokeParam);
if (s32Ret != MI_SUCCESS) {
    printf("Error: MI_IPU_GetInputTensors2 return %d\n", s32Ret);
    return -1;
}
s32Ret = MI_IPU_GetOutputTensors2(u32ChnId, &stInvokeParam);
if (s32Ret != MI_SUCCESS) {
```

```
printf("Error: MI_IPU_ GetOutputTensors2 return %d\n", s32Ret);
MI_IPU_PutInputTensors2(u32ChnId, &stInvokeParam);
return -1;
}

s32Ret = MI_IPU_Invoke2(u32ChnId, &stInvokeParam, &stRuntimeInfo);
if (s32Ret == MI_SUCCESS) {
    // process output buffer data
    // ...
    printf("bw_total=%llu, bw_read=%llu, bw_read=%llu, iputime=%llu\n",
        stRuntimeInfo.u64BandWidth, stRuntimeInfo.u64BandWidthRead,
        stRuntimeInfo.u64BandWidthWrite, stRuntimeInfo.u64IpuTime);
}
```

- Related subject
None.

1.3.16 MI_IPU_Invoke2Custom

- Function
Execute one_buf batch AI network inference.

- Definition
MI_S32 MI_IPU_Invoke2Custom(MI_IPU_CHN u32ChnId, MI_IPU_BatchInvoke_param_t *pstInvokeParam, MI_IPU_RuntimeInfo_t *pstRuntimeInfo);

- Parameter

Parameters	Description	Input/Output
u32ChnId	IPU channel ID	Input
pstInvokeParam	Batch parameter structure pointer	Input
pstRuntimeInfo	IPU operation information structure pointer	Input

- Return value
 - MI_SUCCESS: Successful
 - Others: Failed, see [error code for](#) details
- Dependence
 - Head file: mi_ipu.h

- Library: libmi_ipu.so

➤ Note

- MI_IPU_Invoke2 is synchronous api.
- MI_IPU_Invoke2 can only be used with one_buf mode model.
- The start physical address of each single input/output tensor must be 64 bytes aligned.
- The start physical address of variable tensor which is allocated by user must be 64 bytes aligned.

➤ Example

```
// input tensor number=1 output tensor number=1
MI_IPU_BatchInvoke_param_t stInvokeParam;
MI_IPU_RuntimeInfo_t stRuntimeInfo;

stInvokeParam.u32BatchN = 10;
stInvokeParam.s32TaskPrio = 20;
stInvokeParam.u32IpuAffinity = 0; //Called by ipu
int s32InputSize, s32InputSizeOne;
s32InputSizeOne = stDesc.astMI_InputTensorDescs[0].u32BufSize;
s32InputSize = s32InputSizeOne * stInvokeParam.u32BatchN;

int s32OutputSize, s32OutputSizeOne;
s32OutputSizeOne = stDesc.astMI_OutputTensorDescs[0].u32BufSize;
s32OutputSize = s32OutputSizeOne * stInvokeParam.u32BatchN;
s32Ret = MI_SYS_MMA_Alloc(0, NULL, s32InputSize, &u64InputPhyAddr);
if (s32Ret != MI_SUCCESS) {
    printf("fail to allocate input buffer\n");
    return -1;
}

s32Ret = MI_SYS_Mmap(u64InputPhyAddr, s32InputSize, &pInputVirAddr, TRUE);
if (s32Ret != MI_SUCCESS) {
    MI_SYS_MMA_Free(0, u64InputPhyAddr);
    printf("Error: fail to map input address, error=%d\n", s32Ret);
    return -1;
}

stInvokeParam.astArrayTensors[0].ptTensorData[0] = pInputVirAddr;
```

```

stInvokeParam.astArrayTensors[0].phyTensorAddr[0] = u64InputPhyAddr;

s32Ret = MI_SYS_MMA_Alloc(0, NULL, s32OutputSize, &u64OutputPhyAddr);
if (s32Ret != MI_SUCCESS) {
    MI_SYS_Munmap(pInputVirAddr, s32InputSize);
    MI_SYS_MMA_Free(0, u64InputPhyAddr);
    printf("fail to allocate output buffer\n");
    return -1;
}
s32Ret = MI_SYS_Mmap(u64OutputPhyAddr, s32OutputSize, &pOutputVirAddr, TRUE);
if (s32Ret != MI_SUCCESS) {
    MI_SYS_Munmap(pInputVirAddr, s32InputSize);
    MI_SYS_MMA_Free(0, u64InputPhyAddr);
    MI_SYS_MMA_Free(0, u64OutputPhyAddr);
    printf("Error: fail to map output address, error=%d\n", s32Ret);
    return -1;
}
stInvokeParam.astArrayTensors[1].ptTensorData[0] = pOutputVirAddr;
stInvokeParam.astArrayTensors[1].phyTensorAddr[0] = u64OutputPhyAddr;
for (int i = 0; i < stInvokeParam.u32BatchN; i++) {
    memcpy(pInputVirAddr+i*s32InputSizeOne; pInputBuf[i], s32InputSizeOne);
}
MI_SYS_FlushInvCache(pInputVirAddr, s32InputSize);
s32Ret = MI_IPU_Invoke2Custom(channel, &stInvokeParam, NULL);
if (s32Ret != MI_SUCCESS) {
    printf("fail to invoke\n");
    MI_SYS_Munmap(pInputVirAddr, s32InputSize);
    MI_SYS_MMA_Free(0, u64InputPhyAddr);
    MI_SYS_Munmap(pOutputVirAddr, s32OutputSize);
    MI_SYS_MMA_Free(0, u64OutputPhyAddr);
    return -1;
} else {

```



```
// process output data
for (int i = 0; i < stInvokeParam.u32BatchN; i++) {
    // pOutputVirAddr+i*s32OutputSizeOne
}
}
```

- Related subject
None.

1.3.17 MI_IPU_GetOfflineModeStaticInfo

- Function
Get variable buffer size and offline model file size of an offline model.

- Definition

```
MI_S32 MI_IPU_GetOfflineModeStaticInfo(SerializedReadFunc pReadFunc,
char *pReadCtx, MI_IPU_OfflineModelStaticInfo_t *pStaticInfo);
```

- Parameter

Parameters	Description	Input/Output
pReadFunc	User's custom function pointer to read file (using default file reading function by MI IPU if set NULL)	Input
pReadCtx	Offline model file path	Input
pStaticInfo	Offline model static information structure pointer	Input

- Return value
 - MI_SUCCESS: Successful
 - Others: Failed, see [error code for](#) details

- Dependence
 - Head file: mi_ipu.h
 - Library: libmi_ipu.so

- Example

```
MI_IPU_OfflineModelStaticInfo_t staticInfo;
s32Ret = MI_IPU_GetOfflineModeStaticInfo(NULL, pModelImgPath, &staticInfo);
if (s32Ret == MI_SUCCESS) {
    printf("variable buffer size: %u bytes\n%s size: %u bytes\n",
```

```
staticInfo.u32VariableBufferSize,  
pModelImgPath,  
staticInfo.u32OfflineModelSize);  
}
```

- Related subject
None.

2. DATA TYPES

The table below lists the related data type definition:

Data Types	Definition
SerializedReadFunc	Used for customized file reading mode with customized AI network storage format supported
MI_IPU_ELEMENT_FORMAT	Define enumeration type of IPU input data
MI_IPU_TensorDesc_t	Define IPU Tensor shape structure
MI_IPU_SubNet_InputOutputDesc_t	Define IPU subnet I/O description structure
MI_IPU_Tensor_t	Define IPU Tensor address structure
MI_IPU_TensorVector_t	Define IPU Tensor array structure
MI_IPU_DevAttr_t	Define IPU device attributes structure
MI_IPU_ChnAttr_t	Define IPU channel attributes structure
MI_IPU_BatchInvoke_param_t	Define the batch parameter structure
MI_IPU_RuntimeInfo_t	Define the IPU operation information structure
MI_IPU_OfflineModelStaticInfo_t	Define IPU offline model static information structure

2.1. SerializedReadFunc

➤ Description

Used for customized file reading mode with customized AI network storage format supported.

➤ Definition

```
typedef int (*SerializedReadFunc)(void *dst_buf,int offset, int size, char *ctx);
```

➤ Members

Name	Description
dst_buf	The address of data
offset	The offset from the beginning of file
size	Reading size
ctx	File path

➤ Note

None

➤ Related data type and interface

None

2.2. MI_IPU_ELEMENT_FORMAT

➤ Description

Enumeration type of IPU input data.

➤ Definition

```
typedef enum
{
    MI_IPU_FORMAT_U8,
    MI_IPU_FORMAT_NV12,
    MI_IPU_FORMAT_INT16,
    MI_IPU_FORMAT_INT32,
    MI_IPU_FORMAT_INT8,
    MI_IPU_FORMAT_FP32,
    MI_IPU_FORMAT_UNKNOWN,
    MI_IPU_FORMAT_ARGB8888,
    MI_IPU_FORMAT_ABGR8888,
    MI_IPU_FORMAT_GRAY,
} MI_IPU_ELEMENT_FORMAT;
```

➤ Members

Name	Description
MI_IPU_FORMAT_U8	UINT8 format
MI_IPU_FORMAT_NV12	NV12 format, such as YUV
MI_IPU_FORMAT_INT16	INT16 format
MI_IPU_FORMAT_INT32	INT32 format
MI_IPU_FORMAT_INT8	INT8 format
MI_IPU_FORMAT_FP32	FLOAT format
MI_IPU_FORMAT_UNKNOWN	Unknown
MI_IPU_FORMAT_ARGB8888	ARGB8888 format
MI_IPU_FORMAT_ABGR8888	ABGR8888 format
MI_IPU_FORMAT_GRAY	GRAY format

➤ Note

ARGB/RGB/BGR Tensor belongs to MI_IPU_FORMAT_U8 format.

Only Input Tensor supports MI_IPU_FORMAT_NV12 format.

Only Output Tensor supports MI_IPU_FORMAT_FP32 format.

➤ Related data type and interface

None

2.3. MI_IPU_TensorDesc_t

➤ Description

IPU Tensor description structure.

➤ Definition

```
typedef struct MI_IPU_TensorDesc_s
{
    MI_U32 u32TensorDim;
    MI_IPU_ELEMENT_FORMAT eElmFormat;
    MI_U32 u32TensorShape[MI_IPU_MAX_TENSOR_DIM];
    MI_S8 name[MAX_TENSOR_NAME_LEN];
    MI_U32 u32InnerMostStride;
    MI_FLOAT fScalar;
    MI_S64 s64ZeroPoint;
    MI_S32 s32AlignedBufSize;
    MI_U32 u32BufSize;
    MI_U32 u32InputWidthAlignment;
    MI_U32 u32InputHeightAlignment;
    MI_BOOL bOutputNCHW;
```

```
} MI_IPU_TensorDesc_t;
```

➤ Members

Name	Description
u32TensorDim	Tensor dimension
eElmFormat	Tensor data format
u32TensorShape	Tensor shape array
name	Tensor name
u32InnerMostStride	Tensor inner most dimension's length (unit byte)
fScalar	Tensor quantization coefficient
s64ZeroPoint	Tensor quantization offset
s32AlignedBufSize	Tensor buffer aligned size
u32BufSize	Tensor buffer size
u32InputWidthAlignment	Input tensor aligned size on horizontal direction
u32InputHeightAlignment	Input tensor aligned size on vertical direction
bOutputNCHW	Output tensor is packed on NCHW or not

➤ Note

- The maximum dimension is 10. The following macro definitions are recommended:

```
#define MI_IPU_MAX_TENSOR_DIM (10)
```

- Input data must align to u32InputWidthAlignment and u32InputHeightAlignment, otherwise the result may be incorrect.
- Alignment rules

input_formats	rule
RGB/BGR	No alignment rule
RGBA/BGRA	$W = \text{ALIGN_UP}(W * 4, \text{input_width_alignment}) / 4$ input_width_alignment default is 1
YUV_NV12	$H = \text{ALIGN_UP}(H, \text{input_height_alignment})$ input_height_alignment default is 2 $W = \text{ALIGN_UP}(W, \text{input_width_alignment})$ input_width_alignment default is 2
GRAY	$H = \text{ALIGN_UP}(H, \text{input_height_alignment})$ input_height_alignment default is 1 $W = \text{ALIGN_UP}(W, \text{input_width_alignment})$ input_width_alignment default is 1
RAWDATA_F32_NHWC	No alignment rule
RAWDATA_S16_NHWC	No alignment rule

- Related data type and interface
None

2.4. MI_IPU_SubNet_InputOutputDesc_t

- Description
IPU subnet I/O description structure.
- Definition

```
typedef struct MI_IPU_SubNet_InputOutputDesc_s
{
    MI_U32 u32InputTensorCount;
    MI_U32 u32OutputTensorCount;
    MI_IPU_TensorDesc_t astMI_InputTensorDescs[MI_IPU_MAX_INPUT_TENSOR_CNT];
    MI_IPU_TensorDesc_t astMI_OutputTensorDescs[MI_IPU_MAX_OUTPUT_TENSOR_CNT];
} MI_IPU_SubNet_InputOutputDesc_t;
```
- Members

Name	Description
u32InputTensorCount	Number of input tensor
u32OutputTensorCount	Number of output tensor
astMI_InputTensorDescs	Input Tensor shape structure array
astMI_OutputTensorDescs	Output Tensor shape structure array
- Note
None
- Related data type and interface
None

2.5. MI_IPU_Tensor_t

- Description
IPU Tensor address structure.
- Definition

```
typedef struct MI_IPU_Tensor_s
{
```

```
void    *ptTensorData[2];
MI_PHY phyTensorAddr[2]; //notice that this is miu bus addr, not cpu bus addr.
} MI_IPU_Tensor_t;
```

➤ Members

Name	Description
ptTensorData	The virtual address of Tensor buffer
phyTensorAddr	The physical address of Tensor buffer

➤ Note

None

➤ Related data type and interface

None

2.6. MI_IPU_TensorVector_t

➤ Description

IPU Tensor array structure.

➤ Definition

```
typedef struct MI_IPU_TensorVector_s
{
    MI_U32 u32TensorCount;
    MI_IPU_Tensor_t astArrayTensors[MI_IPU_MAX_INPUT_TENSOR_CNT];
} MI_IPU_TensorVector_t;
```


➤ Members

Name	Description
u32TensorCount	The number of Tensor
astArrayTensors	Address information of each Tensor

➤ Note

None

➤ Related data type and interface

None

2.7. MI_IPU_DevAttr_t

➤ Description

IPU device attributes structure.

➤ Definition

```
typedef struct MI_IPU_DevAttr_s
{
    MI_U32 u32MaxVariableBufSize;
    MI_U32 u32YUV420_W_Pitch_Alignment;
    MI_U32 u32YUV420_H_Pitch_Alignment;
    MI_U32 u32XRGB_W_Pitch_Alignment;
} MI_IPU_DevAttr_t;
```

➤ Members

Name	Description
u32MaxVariableBufSize	Maximum memory cost of IPU
u32YUV420_W_Pitch_Alignment	Pitch size of YUV horizontal direction (only useful when input format is YUV)
u32YUV420_H_Pitch_Alignment	Pitch size of YUV vertical direction (only useful when input format is YUV)
u32XRGB_W_Pitch_Alignment	Pitch size of RGBA/BGRA horizontal direction (only useful when input format is RGBA/BGRA)

➤ Note

None

➤ Related data type and interface

None

2.8. MI_IPU_ChAttr_t

➤ Description

IPU channel attributes structure.

➤ Definition

```
typedef struct MI_IPU_ChAttr_s
{
    MI_U32 u32SubNetId;
    MI_U32 u32OutputBufDepth;
    MI_U32 u32InputBufDepth;
    MI_U32 u32BatchMax;
} MI_IPUChnAttr_t;
```

➤ Members

Name	Description
u32SubNetId	Subnetwork ID
u32OutputBufDepth	The depth of output tensor buffer
u32InputBufDepth	The depth of input tensor buffer
u32BatchMax	Maximum batch size

➤ Note

Maximum depth of IPU I/O buffer is 3. The following macro definitions are recommended:

```
#define MAX_IPU_INPUT_OUTPUT_BUF_DEPTH (3)
```

➤ Related data type and interface

None

2.9. MI_IPU_BatchInvoke_param_t

➤ Description

Define the batch parameter structure.

➤ Definition

```
typedef struct MI_IPU_BatchInvoke_param_s {
    MI_PHY u64VarBufPhyAddr;
    MI_U32 u32VarBufSize;
    MI_U32 u32BatchN;
    MI_S32 s32TaskPrio;
    MI_U32 u32IpuAffinity;
    MI_IPU_Tensor_t astArrayTensors[MI_IPU_MAX_BATCH_TENSOR_CNT];
} MI_IPU_BatchInvoke_param_t;
```


➤ Members

Name	Description
u64VarBufPhyAddr	Variable buffer physical address allocated by user
u32VarBufSize	Variable buffer size allocated by user
u32BatchN	Number of batches
s32TaskPrio	Task priority
u32IpuAffinity	Binding ipu core
astArrayTensors	Batch process all input and output tensor buffer addresses

➤ Note

- The astArrayTensors array stores all input and output tensor buffer addresses of batch processing. The rule is to store all input tensor buffer addresses in turn, and then store all output tensor buffer addresses.
- The start physical address of each single input/output tensor must be 64 bytes aligned.
- The start physical address of variable tensor which is allocated by user must be 64 bytes aligned.

➤ Related data type and interface

None

2.10. MI_IPU_RuntimeInfo_t

➤ Description

Define the IPU operation information structure.

➤ Definition

```
typedef struct MI_IPU_RuntimeInfo_s {
    MI_U64 u64BandWidth;
    MI_U64 u64IpuTime;
    MI_U64 u64BandWidthRead;
    MI_U64 u64BandWidthWrite;
} MI_IPU_RuntimeInfo_t;
```

➤ Members

Name	Description
u64BandWidth	Total bandwidth
u64IpuTime	IPU processing time
u64BandWidthRead	Bandwidth of reading
u64BandWidthWrite	Bandwidth of writing

➤ Note

None

- Related data type and interface
None

2.11. [MI_IPU_OfflineModelStaticInfo_t](#)

- Description
Define IPU offline model static information structure.

- Definition

```
typedef struct MI_IPU_OfflineModelStaticInfo_s {  
    MI_U32 u32VariableBufferSize;  
    MI_U32 u32OfflineModelSize;  
} MI_IPU_OfflineModelStaticInfo_t;
```

- Members

Name	Description
u32VariableBufferSize	Variable buffer size for running the offline model
u32OfflineModelSize	Offline model file size

- Note
None

- Related data type and interface
None

3. ERROR CODES

Error Code	Definition	Description
0	MI_SUCCESS	Success
1	E_IPU_ERR_INVALID_CHNID	Invalid channel ID
2	E_IPU_ERR_CHNID_EXIST	Channel already exists
3	E_IPU_ERR_CHNID_UNEXIST	Channel does not exist
4	E_IPU_ERR_NOMEM	Failure caused by malloc memory
5	E_IPU_ERR_NOBUF	Failure caused by malloc buffer
6	E_IPU_ERR_BADADDR	Bad address, buffer address is not gotten from IPU buffer allocator
7	E_IPU_ERR_SYS_TIMEOUT	System timeout
8	E_IPU_ERR_FILE_OPERATION	File cannot be opened or read or written
9	E_IPU_ERR_ILLEGAL_TENSOR_BUFFER_SIZE	Tensor buffer size does not meet the requirement, usually less than the requirement
10	E_IPU_ERR_ILLEGAL_BUFFER_DEPTH	Input or output buffer depth quantum exceeds maximum number
11	E_IPU_ERR_ILLEGAL_INPUT_OUTPUT_DESC	Network description is illegal, usually input or output buffer quantum is wrong
12	E_IPU_ERR_ILLEGAL_INPUT_OUTPUT_PARAM	User's input or output buffer quantum does not match network description
13	E_IPU_ERR_MAP	Address mapping error
14	E_IPU_ERR_INIT_FIRMWARE	Fail to initialize IPU firmware
15	E_IPU_ERR_CREATE_CHANNEL	Fail to create channel
16	E_IPU_ERR_DESTROY_CHANNEL	Fail to destroy channel
17	E_IPU_ERR_INVOKE	Fail to invoke
18	E_IPU_ERR_SET_MALLOC_REGION	Fail to set malloc region for freertos
19	E_IPU_ERR_SET_IPU_PARAMETER	Fail to set IPU parameter
20	E_IPU_ERR_INVALID_PITCH_ALIGNMENT	Invalid pitch alignment
21	E_IPU_ERR_NO_CREATED_IPU_DEVICE	There is no created IPU device
22	E_IPU_ERR_GET_IPU_VERSION	Fail to get IPU version from IPU firmware
23	E_IPU_ERR_MISMATCH_IPU_HEAD_FILE	IPU head files version not matched

Error Code	Definition	Description
24	E_IPU_ERR_NO_SUPPORT_REQ	IPU firmware does not support this request
25	E_IPU_ERR_FAILED	Unexpected error
26	E_IPU_ERR_SEND_REQUEST	Fail to send request to IPU
27	E_IPU_ERR_GET_FIRMWARE_INFO	Fail to get IPU firmware information
28	E_IPU_ERR_INVALID_IPUCORE_BOOTING_PARAMETERS	Invalid IPU cores booting parameters
29	E_IPU_ERR_INVALID_IPUCORE_SHUTDOWN_PARAMS	Invalid IPU cores shutdowning parameters
30	E_IPU_ERR_NO_MULTICORE_ENV	Multicore mode needs all ipu cores being alive
31	E_IPU_ERR_INVALID_TASK_PRIORITY	Invalid ipu task priority
32	E_IPU_ERR_DEV_SHUTDOWN	Ipu core has been shutdown
33	E_IPU_ERR_DEV_FAIL_RESET	Fail to reset ipu
34	E_IPU_ERR_DEV_FAIL_SHUTDOWN	Fail to shutdown ipu
35	E_IPU_ERR_NO_AVAILABLE_DEV	No available ipu dev
36	E_IPU_ERR_RESET_OFF	Reset function is off
37	E_IPU_ERR_INVALID_BATCH_NUM	batch number error
38	E_IPU_ERR_BATCH_TYPE	batch type error
39	E_IPU_ERR_BATCH_MODE	batch mode error
40	E_IPU_ERR_NO_AVAILABLE_BATCH_MODE	do not find available batch mode
41	E_IPU_ERR_IPU_HANG	invoke was dropped due to ipu hang
42	E_IPU_ERR_NO_RESET_DEV	no reset ipu dev
43	E_IPU_ERR_NO_BATCH_PARAM	no batch parameter
44	E_IPU_ERR_INVALID_MODEL_BUFFER	invalid user model buffer physical address or size
45	E_IPU_ERR_INVALID_VARIABLE_BUFFER	invalid variable buffer physical address or size
46	E_IPU_ERR_NOT_ASSIGN_CORE	not assign ipu core when use user's variable buffer
47	E_IPU_ERR_SWDISP_NOT_REGISTER	model has unsupported swdisp function
48	E_IPU_ERR_SWDISP_NOT_FIND_TASKID	not find swdisp task id
49	E_IPU_ERR_SWDISP_INVALID_PARAM	invalid swdisp parameter
50	E_IPU_ERR_SWDISP_UNEXPECTED	unexpected swdisp error
51	E_IPU_ERR_SWDISP_UNKNOWN	unknown swdisp error
52	E_IPU_ERR_BAD_PHY_ADDR_ALIGNMENT	ipu buffer physical addr not aligned
53	E_IPU_ERR_MISMATCH_INVOKE_FUNC	n_buf/one_buf batch model should use MI_IPU_Invoke2/MI_IPU_Invoke2Custom

Error Code	Definition	Description
256	E_IPU_ERR_NO_AVAILABLE_CHNID	There is no available channel

4. PROCFS INTRODUCTION

4.1. Summary

The main ways to debug IPU through the console are procfs and sysfs.

- IPU Procfs creates the node "proc/mi_module/mi_ipu/mi_ipu0" when opening the device (mi_dev), and deletes the node when closing the device.
- IPU Sysfs creates the node "sys/dla/xxx" when insmod ko.

4.2. How to cat MMA info

Command: cat /proc/mi_modules/mi_sys/mi_sys0

```
/ # cat /proc/mi_modules/mi_sys/mi_sys0
```

```
-----mma allocator info for DevId:0-----
mma_heap_name0      30180000      length      fe40000      chunk_mgr_avail      f326000      mmuEnable      1      addsize_before      0      addsize_after      0      bMmuDelayUnmap      0
chunk_mgr_info:      offset      length      avail
0      10000000      f4e6000
each chunk info:
offset      length      used_flag      task_name      pid      Module
0      c0000      1      CMDMEM      -1      mi_sys
c0000      1000      1      sys-logConfig      -1      mi_sys
c1000      40000      1      sys-logBuffer      -1      mi_sys
101000      1a1000      1      ipu_variable      -1      mi_ipu
2a2000      400000      1      ipu_heap      -1      mi_ipu
6a2000      2000      1      IPU log      -1      mi_ipu
6a4000      407000      1      ipu_chn0_networ      720      mi_ipu
aab000      a000      1      ipu_chn0_desc      720      mi_ipu
ab5000      1000      1      IPU Buffer_chn0      720      mi_ipu
ab6000      31000      1      ipu_chn0_input_      720      mi_ipu
ae7000      31000      1      ipu_chn0_input_      720      mi_ipu
b18000      1000      1      ipu_chn0_output      720      mi_ipu
b19000      1000      1      ipu_chn0_output      720      mi_ipu
b1a000      f4e6000      0      ipu_chn0_output      -1      mi_sys
mma_heap_ipu      2ffe0000      164000      0      0      0
chunk_mgr_info:      offset      length      avail
0      164000      0
each chunk info:
offset      length      used_flag      task_name      pid      Module
0      164000      1      ipu_firmware      -1      mi_ipu
imi_mma_heap      20000000      43800      43800      0      0
chunk_mgr_info:      offset      length      avail
0      43800      43800
each chunk info:
offset      length      used_flag      task_name      pid      Module
0      43800      0      NA      -1      mi_sys
```

You can see the usage of all MMAs.

Command: cat /proc/mi_modules/mi_ipu/mi_ipu0

```
/ # cat /proc/mi_modules/mi_ipu/mi_ipu0
```

```
-----CMDQ Kickoff counter-----
DevId  current_buf_size  Peak_buf_size
0      707000           70b000
each dev buf info:
offset      length      used_flag      task_name
101000      1a1000      1              ipu_variable
2a2000      400000      1              ipu_heap
6a2000      2000       1              IPU_log
0           164000      1              ipu_firmware

-----Common info formi_ipu only dump enabled chn-----
ChnId PassNum  EnInPNum  EnOutPNum  MMAHeapName
0      0        0          0          (null)
ChnId  current_buf_size  Peak_buf_size  user_pid
0      476000          476000         720
each chn buf info:
offset      length      used_flag      task_name
6a4000      407000      1              ipu_chn0_networ
aab000      a000       1              ipu_chn0_desc
ab5000      1000       1              IPU_Buffer_chn0
ab6000      31000      1              ipu_chn0_input_
ae7000      31000      1              ipu_chn0_input_
b18000      1000       1              ipu_chn0_output
b19000      1000       1              ipu_chn0_output
```

You can see how the IPU device and each channel use MMA.

4.3. Cat IPU version info

Command: `echo version > /proc/mi_modules/mi_ipu/mi_ipu0`

```
/ # echo version > /proc/mi_modules/mi_ipu/mi_ipu0
lib version: P_1.0.1_210525-86ece6aa9a0cfe21fd30126fcb92b183
driver version: P_1.0.1_210525
```

Command: `cat /sys/dla/version`

```
/ # cat /sys/dla/version
lib version: P_1.0.1_210525-86ece6aa9a0cfe21fd30126fcb92b183
driver version: P_1.0.1_210525
```

4.4. Cat IPU clock info

Command: `cat /sys/dla/freq`

```
/ # cat /sys/dla/freq
current ipu clock frequency: 800MHz
available frequency:
1500MHz
1400MHz
1300MHz
1200MHz
1100MHz
1000MHz
900MHz
800MHz
700MHz
600MHz
500MHz
400MHz
300MHz
200MHz
150MHz
```

4.5. Adjust IPU clock

Command: `echo xxx > /sys/dla/freq`

(XXX must be available frequency)

```
/ # echo 900 > /sys/dla/freq
/ # cat /sys/dla/freq
current ipu clock frequency: 900MHz
available frequency:
1500MHz
1400MHz
1300MHz
1200MHz
1100MHz
1000MHz
900MHz
800MHz
700MHz
600MHz
500MHz
400MHz
300MHz
200MHz
150MHz
```

4.6. Switch auto reset function

The Auto reset function can reset the IPU after no response from the IPU and continue unfinished tasks.

Turn on auto reset function: `echo on > /sys/dla/auto_reset`

Turn off auto reset function: `echo off > /sys/dla/auto_reset`

In the early testing stage, it is recommended to turn off the auto reset function to clarify the reason why the IPU is not responding.

4.7. Get IPU log

Command:

```
echo "ctrl_size=0x800000 corectrl_size=0x800000 ctrl=0xffffffff corectrl=0x1fff" > /sys/dla/ipu_log
```

ctrl_size and corectrl_size are the buffer sizes allocated for ctrl log and corectrl log, ctrl and corectrl are the configuration of ctrl log and corectrl log.